

Monitores:

Resolver el siguiente problema con **MONITORES**. Simular la atención en una Salita Médica para vacunar contra el coronavirus. Hay **UNA enfermera** encargada de vacunar a **30 pacientes**, cada paciente tiene un turno asignado (valor entero entre 1..30 ya conocido por el paciente). La enfermera atiende a los pacientes de acuerdo al turno que cada uno tiene asignado. Cada paciente al llegar espera a que sea su turno y se dirige al consultorio para que la enfermera lo vacune, y luego se retira. **Nota:** suponer que existe una función *Vacunar()* que simula la atención del paciente por parte de la enfermera. Todos los procesos deben terminar.

Para que quede más clara la explicación supondremos que los turnos son de 0..29

```
Process Enfermera {  
    int cant;  
    for (cant = 0; cant < 30; cant++) {  
        Salita.Siguiente();  
        Consultorio.EsperarPaciente();  
        Vacunar();  
        Consultorio.TerminarVacunacion();  
    }  
}
```

```
Process Paciente[id=0..29] {  
    int turno = .....;  
  
    Salita.Esperar (turno);  
    Consultorio.EsperarAtencion();  
}
```

```
MONITOR Salita {  
    int turnoActual = -1;  
    cond espera[30], enf;  
    bool presente[30] = ([30] false);  
  
    Procedure Esperar (int turno) {  
        presente[turno] = true;  
        if (turnoActual == turno) signal (enf);  
        wait (espera[turno]);  
    }  
  
    Procedure Siguiente () {  
        turnoActual++;  
        if (not presente[turnoActual]) wait (enf);  
        signal (espera[turnoActual]);  
    }  
}
```

```
MONITOR Consultorio {  
    bool llegoPaciente = false;  
    cond enf, pac;  
  
    Procedure EsperarAtencion () {  
        llegoPaciente = true;  
        signal (enf);  
        wait (pac);  
        signal (enf);  
    }  
  
    Procedure EsperarPaciente() {  
        if (not llegoPaciente) wait (enf);  
        llegoPaciente = false;  
    }  
  
    Procedure TerminarVacunacion() {  
        signal (pac);  
        wait (enf);  
    }  
}
```

Resolver con **MONITORES** el siguiente problema. Simular el uso de un puente de un solo carril y sentido por donde puede pasar un vehículo a la vez. Hay ***N vehículos*** donde algunos son autos y otros ambulancias. Los vehículos deben pasar de acuerdo al orden de llegada pero siempre dando prioridad a las ambulancias. ***Nota:*** suponga que cada vehículo tarda 5 minutos en pasar por el puente.

```
Process Vehiculo[id: 0..N] {  
    boolean esAmbulancia = ....;  
  
    if (esAmbulancia) Admin.PasoAmbulancia()  
    else Admin.PasoAuto();  
    // pasa por el puente  
    delay(300);  
    Admin.Salir();  
};
```

```
Monitor Admin {  
    bool libre = true;  
    int cantAmbulancia = 0, cantAutos = 0;  
    cond esperaAuto, esperaAmbulancia;  
  
    Procedure PasoAmbulancia{  
        if (libre) libre = false  
        else { cantAmbulancia++;  
              wait (esperaAmbulancia);  
            };  
    };  
  
    Procedure PasoAuto{  
        if (libre) libre = false  
        else { cantAuto++;  
              wait (esperaAuto);  
            };  
    };  
  
    Procedure Salir{  
        if (cantAmbulancia > 0) { cantAmbulancia--;  
                                   signal(esperaAmbulancia);  
                                }  
        else if (cantAutos > 0) { cantAutos--;  
                                   signal(esperaAuto);  
                                }  
        else libre = true;  
    };  
};
```

Resolver con **MONITORES** el siguiente problema. Se debe simular una carrera a campo traviesa con C *corredores* donde en la mitad del recorrido hay un puente colgante que puede ser usado por una única persona a la vez. Cuando los C *corredores* han llegado al punto de partida comienza la carrera. Cuando un corredor llega al puente espera su turno (respetando el orden de llegada al mismo) y lo cruza (suponga que tarda un par de minutos en cruzarlo); y luego continua su carrera hasta llegar a la meta. **Nota:** cada corredor pasa sólo una vez por el puente. Sólo se pueden usar procesos que representen a los corredores.

Process Corredor[id: 0..C-1] {

Largada.EsperarInicio();

Correr;

AdminPuente.Solicitar();

Pasa por el puente

AdminPuente.Dejar();

Correr;

};

Monitor Largada{

int cantidad = 0;

cond espera;

Procedure EsperarInicio() {

cantidad++;

if (cantidad < C) wait (espera)

else signal_all (espera);

};

}

Monitor AdminPuente {

bool libre = true;

int cant = 0;

cond espera;

Procedure Solicitar {

if (libre) libre = false

else { cant++;

wait (espera);

};

};

Procedure Dejar {

if (cant == 0) libre = true

else { cant--;

signal(espera);

};

};

}

Resolver el siguiente problema con **MONITORES**. Simular la atención en un Centro de Vacunación con 8 puestos para vacunar contra el coronavirus. Al Centro acuden **200 pacientes** para ser vacunados, cada uno de ellos ya conoce el puesto al que se debe dirigir. En cada puesto hay **UN empleado** para vacunar a los pacientes asignados a dicho puesto, y lo hace de acuerdo al orden dado por la edad de paciente (cuando está libre atiende al paciente de mayor edad que esté esperando en ese momento en ese puesto). Cada paciente al llegar al puesto que tenía asignado espera a que lo llamen y se dirige a la silla para que el empleado lo vacune, y luego se retira. **Nota:** suponer que existe una función *Vacunar()* que simula la atención del paciente. Suponer que cada puesto tiene asignado 25 pacientes. Todos los procesos deben terminar.

```

Proces Empleado[id: 0..7]{
    int cant;
    for (cant = 0; cant < 25; cant++) {
        Puesto[id].Siguierte();
        Silla[id].EsperarPaciente();
        Vacunar();
        Silla[id].TerminarVacunacion();
    }
}

```

```

Process Paciente[id=0..199] {
    int numP = .....;
    int edad = ....;

    Puesto[numP].Esperar (id, edad);
    Silla[numP].EsperarAtencion();
}

```

```

MONITOR Puesto[id: 0..7] {
    cond esperaPac[200], emp;
    colaOrdenada cola;
    int idPac, e;

    Procedure Esperar (int idP, int edad) {
        agregar(col, (idP, edad));
        signal (emp);
        wait (esperaPac[idP]);
    }

    Procedure Siguierte () {
        if (empty(col)) wait (emp);
        sacar (col, (idPac, e));
        signal (esperaPac[idPac]);
    }
}

```

```

MONITOR Silla [id: 0..7] {
    bool llegoPaciente = false;
    cond emp, pac;

    Procedure EsperarAtencion () {
        llegoPaciente = true;
        signal (emp);
        wait (pac);
        signal (emp);
    }

    Procedure EsperarPaciente() {
        if (not llegoPaciente) wait (emp);
        llegoPaciente = false;
    }

    Procedure TerminarVacunacion() {
        signal (pac);
        wait (emp);
    }
}

```


Resolver con **MONITORES** el siguiente problema. En una sala se juntan **20 conferencistas** y **un coordinador** para una conferencia internacional. Cuando todos han llegado a la sala (los 20 conferencistas y el coordinador) el coordinador abre la sesión con una presentación de 30 minutos, y luego cada conferencista realiza su presentación de 10 minutos, de a uno a la vez y de acuerdo al orden en que llegaron a la sala. Cuando todas las presentaciones terminaron, las personas (conferencistas y coordinador) se retiran.

```
Process Coordinador {  
    Sala.EsperarInicio();  
    Hace presentación de 30 minutos;  
    Sala.Terminar();  
};
```

```
Process Conferencista[id: 0..19]{  
    Sala.EsperarTurno();  
    Hace presentación de 10 minutos;  
    Sala.Terminar();  
};
```

```
Monitor Sala {  
    int cantidad = 0, terminaron = 0;  
    cond esperaCoordinador, espera, esperaFin;  
  
    Procedure EsperarInicio() {  
        if (cantidad < 20) wait (esperaCoordinador);  
    };  
  
    Procedure EsperarTurno() {  
        cantidad++;  
        if (cantidad == 20) signal (esperaCoordinador);  
        wait (espera);  
    };  
  
    Procedure Terminar() {  
        terminaron++;  
        if (terminaron == 21) signal_all (esperaFin)  
        else { signal (espera);  
              wait (esperaFin);  
            };  
    };  
}
```

Resolver con **Monitores** el siguiente problema. En un parque hay un juego para ser usada por N personas de a una a la vez y de acuerdo al orden en que llegan para solicitar su uso. Además hay un empleado encargado de desinfectar el juego durante 10 minutos antes de que una persona lo use. Cada persona al llegar espera hasta que el empleado le avisa que puede usar el juego, lo usa por un tiempo y luego lo devuelve. **Nota:** suponga que la persona tiene una función *Usar_juego* que simula el uso del juego; y el empleado una función *Desinfectar_Juego* que simula su trabajo.

Process Persona [id: 0..N-1] {

Admin.SolicitarUso ();

Usar_Juego ();

Admin.Liberar ();

}

Process Empleado {

while (true) {

Desinfectar_Juego ();

Admin.Listo ();

}

}

MONITOR Admin {

cond ePersona, eEmpleado;

int cant = 0;

bool listo = false;

Procedure SolicitarUso () {

if (listo) listo = false

else { cant++;

wait (ePersona);

};

}

Procedure Liberar () {

signal (eEmpleado);

}

Procedure Listo () {

if (cant > 0) { cant--;

signal (ePersona);

}

else listo = true;

}

}

Resolver con **Monitores** el siguiente problema. En un gimnasio para rehabilitación hay una máquina que debe ser usada por N personas, de a una a la vez y de acuerdo al orden dado por la edad (*cuando la máquina está libre la debe usar la de mayor edad que está esperando usarla*). **Nota:** suponga que la persona tiene una función *Usar_Máquina* que simula el uso de la máquina.

```
Process Persona [id: 0..N-1] {  
    int edad = ...;  
  
    Admin.SolicitarUso (id, edad);  
    Usar_Máquina ();  
    Admin.Liberar ();  
}
```

```
MONITOR Admin {  
    cond espera[N];  
    int idAux, cant = 0;  
    colaOrdenada c;  
    bool libre = true;  
  
    Procedure SolicitarUso (int idP, int E) {  
        if (libre) libre = false  
        else { cant++;  
              agregar (c, (idP, E));  
              wait (espera[idP]);  
            };  
    }  
  
    Procedure Liberar () {  
        if (cant > 0) { cant--;  
                      idAux = sacar (c);  
                      signal (espera[idAux]);  
                      }  
        else libre = true;  
    }  
}
```


Resolver con **Monitores** el siguiente problema. Hay N personas que deben usar un teléfono público de a una a la vez y de acuerdo al orden de llegada, pero dando prioridad a las que tienen que usarlo con urgencia (*cada persona ya sabe al comenzar si es de tipo urgente o no*). **Nota:** suponga que la persona tiene una función *Usar_Teléfono* que simula el uso del teléfono.

```
Process Persona [id: 0.. $N$ -1] {  
    bool urgente=...;  
  
    Admin.SolicitarUso (urgente);  
    Usar_Teléfono ();  
    Admin.Liberar ();  
}
```

```
MONITOR Admin {  
    cond eUrgente, eNormal;  
    int cantU = 0, cantN = 0;  
    bool libre = true;  
  
    Procedure SolicitarUso (bool prioridad) {  
        if (libre) libre = false  
        else if (prioridad){ cantU++;  
                           wait (eUrgente);  
                           }  
        else { cantN++;  
              wait (eNormal);  
              }  
    };  
}  
  
    Procedure Liberar () {  
        if (cantU > 0) { cantU--;  
                       signal (eUrgente);  
                       }  
        else if (cantN > 0) { cantN--;  
                           signal (eNormal);  
                           }  
        else libre = true;  
    }  
}
```

Resolver el siguiente problema con **MONITORES**. En un examen de la secundaria hay *un preceptor* y *una profesora* que deben tomar un examen escrito a **45 alumnos**. El preceptor se encarga de darle el enunciado del examen a los alumnos cuando los 45 han llegado (es el mismo enunciado para todos). La profesora se encarga de ir corrigiendo los exámenes de acuerdo al orden en que los alumnos van entregando. Cada alumno al llegar espera a que le den el enunciado, resuelve el examen, y al terminar lo deja para que la profesora lo corrija y le dé la nota. **Nota:** maximizar la concurrencia; todos los procesos deben terminar su ejecución; suponga que la profesora tienen una función *corregirExamen* que recibe un examen y devuelve un entero con la nota.

```

Process Alumno [id: 0..44] {
    int nota;
    text examen;
    Aula.Llegada (examen);
    // hace el examen
    Aula.Entrega (id, examen, nota);
}

```

```

Process Profesora {
    int i, idA, nota;
    text examen;
    for (i=0; i<45; i++) {
        Aula.Examen (idA, examen);
        nota = corregirExamen (examen);
        Aula.Corregido (idA, nota);
    }
}

```

```

Process Preceptor {
    text enunciado = .....;
    Aula.DarEnunciado (enunciado);
}

```

```

Monitor Aula {
    int cant=0, notas[45];
    text enunciado;
    cola C;
    cond eInicio, ePreceptor, eProfesora;
    cond eNota;
}

```

```

Procedure Llegada (E: OUT text)
{
    cant++;
    if (cant == 45) signal (ePreceptor);
    wait (eInicio);
    E = enunciado;
}

```

```

Procedure Entrega (ID: int, E: text, N: out int)
{
    push (C, (ID, E));
    signal (eProfesora);
    wait (eNota);
    N = notas[ID];
}

```

```

Procedure DarEnunciado (E: text)
{
    if (cant < 45) wait (ePreceptor);
    enunciado = E;
    signal_all (eInicio);
}

```

```

Procedure Examen (idA: out int; E: out text)
{
    if (empty(C)) wait (eProfesora);
    pop (C, (idA, E));
}

```

```

Procedure Corregido (ID: int, N: int)
{
    notas[id] = N;
    signal (eNota);
}

```

PMA

Resolver con **PMA (Pasaje de Mensajes ASINCRÓNICOS)** el siguiente problema. En una oficina hay **3 empleados** y **P personas** que van para ser atendidas para realizar un trámite. Cuando una persona llega espera hasta ser atendido por cualquiera de los empleados, le indica el trámite a realizar y espera a que le den el resultado. Los empleados atienden las solicitudes en orden de llegada; si no hay personas esperando, durante 5 minutos resuelven trámites pendientes (simular el proceso de resolver trámites pendientes por medio de un *delay*). **Nota:** no generar demora innecesaria; cada persona hace sólo un pedido y termina; los empleados no necesitan terminar su ejecución.

```
chan solicitud (int);
chan respuestaTramite[P] (text);
chan respuesta[P] (int);
chan siguiente (int);
chan respuestaEmpleado[3] (int);
chan datosTramite[3] (text);
```

```
Process Empleado[id: 0..2] {
  int idP;
  text tramite, res;
  while (true) {
    send siguiente (id);
    receive respuestaEmpleado[id] (idP);
    if (idP > -1) {
      send respuesta[idP] (id);
      receive datosTramite[id] (tramite);
      respuesta = resolverTramite(tramite);
      send respuestaTramite[idP] (res);
    }
    else delay (5 minutos);
  }
};
```

```
Process Persona[id: 0..P-1] {
  text res, tramite;
  int idE;

  send solicitud(id);
  receive respuesta[id] (idE);
  send datosTramite[idE] (tramite);
  receive respuestaTramite[id] (res);
};
```

```
Process Admin {
  int idP, idE;

  while (true) {
    receive siguiente(idE);
    if (not empty (solicitud)) receive solicitud(idP)
    else idP = -1;

    send respuestaEmpleado[idE] (idP);
  }
};
```


Resolver con **PMA (Pasaje de Mensajes ASINCRÓNICOS)** el siguiente problema. En un negocio hay **5 empleados** que atienden a ***N personas*** que van a pedir un presupuesto, de acuerdo al orden de llegada. Cuando el cliente sabe que empleado lo va a atender le entrega el listado de productos que necesita, y luego el empleado le entrega el presupuesto del mismo. **Nota:** maximizar la concurrencia. Existe una función *HacerPresupuesto(lista)* que simula la elaboración del presupuesto por parte de los empleados .

```
chan solicitud (int);
chan respuesta[N] (int);
chan datosListado[5] (text);
chan respuestaPresupuesto[N] (text);
```

```
Process Persona[id: 0..N-1] {
    text lista, res;
    int idE;

    send solicitud(id);
    receive respuesta[id] (idE);
    send datosListado[idE] (lista);
    receive respuestaPresupuesto[id] (res);
};
```

```
Process Empleado[id: 0..4] {
    int idP;
    text listado, resultado;

    while (true) {
        receive solicitud (idP);
        send respuesta[idP] (id);
        receive datosListado[id] (listado);
        resultado = HacerPresupuesto (listado);
        send respuestaPresupuesto[idP] (resultado);
    };
};
```

Resolver el siguiente problema con **Pasaje de Mensajes Asíncronos (PMA)**. En una empresa de software hay **3 programadores** que deben arreglar errores informados por ***N clientes***. Los clientes continuamente están trabajando, y cuando encuentran un error envían un reporte a la empresa para que lo corrija (no tienen que esperar a que se resuelva). Los programadores resuelven los reclamos de acuerdo al orden de llegada, y si no hay reclamos pendientes trabajan durante una hora en otros programas. **Nota:** los procesos no deben terminar (trabajan en un loop infinito); suponga que hay una función *ResolverError* que simula que un programador está resolviendo un reporte de un cliente, y otra *Programar* que simula que está trabajando en otro programa.

```
chan entrega (text);  
chan pedir (int);  
chan siguiente[3] (text);
```

```
Process Admin {  
    int idP;  
    text r;  
  
    while (true) {  
        receive pedir (idP);  
        if (empty(entrega)) r = NULL  
        else receive entrega(r);  
        send siguiente[idP] (r);  
    }  
};
```

```
Process Cliente [id: 0..N-1] {  
    text reporte;  
    while (true) {  
        //trabaja  
        reporte = DetectaError();  
        send entrega (reporte);  
    }  
};
```

```
Process Programador [id: 0..2] {  
    text R;  
    while (true) {  
        send pedir(id);  
        receive siguiente[id] (R);  
        if (R <> NULL) ResolverError (R)  
        else Programar();  
    }  
};
```

Resolver con **PASAJE DE MENSAJES ASINCRÓNICOS (PMA)** el siguiente problema. Se debe simular la atención en un peaje con **7 cabinas** para atender a **N vehículos** (algunos de ellos son **ambulancias**). Cuando el vehículo llega al peaje se dirige a la cabina con menos vehículos esperando y se queda ahí hasta que lo terminan de atender y le dan el ticket de pago. Las cabinas atienden a los vehículos que van a ella de acuerdo al orden de llegada pero dando prioridad a las ambulancias; cuando terminan de atender a un vehículo le dan el ticket de pago. **Nota:** maximizar la concurrencia.

```
chan pedirNumC(int), darNumC[N](int);
chan salir(int), avisoA();
chan llegaA[7](int), llegaC[7](int);
chan avisoV[7](), libre[7]();
chan darTicket[N](text);
```

Process Vehiculo[id: 0..N-1]

```
{ int numCab;
  text miTicket;

  send pedirNumC(id);
  send avisoA();
  receive darNumC[id] (numCab);
  if (es Ambulancia) send llegaA[numCab](id)
  else send llegaC[numCab] (id);
  send avisoV[numCab]();
  receive darTicket[id] (miTicket);
  -pasa la cabina
  send libre[numCab]();
  send salir (numCab);
  send avisoA();
}
```

Process Cabina[id: 0..6]

```
{ int idV;    text ticket;

  while (true) {
    receive avisoV[id]();
    if (not empty(llegaA[id])) receive llegaA[id](idV)
    else receive llegaC[id](idV);
    ticket = generarTicket;
    send darTicket[idV] (ticket);
    receive libre[id]();
  }
}
```

Process Admin

```
{ int idV, numC, cant[7] = ([7] 0);

  while (true) {
    receive avisoA();
    if (empty(salir) && (not empty(pedirNumC)) → receive pedirNumC(idV);
                                           numC = posicionMinimo (cant);
                                           send darNumC[idV] (numC);
                                           cant[numC]++;

    □ (not empty(salir)) → receive salir(numC);
                           cant[numC]--;

    fi
  }
}
```

Resolver con **PASAJE DE MENSAJES ASINCRÓNICOS (PMA)** el siguiente problema. Se debe simular la atención en una planta para la VTV con **5 puestos** para atender a **N vehículos** (que pueden ser **sanitarios** o **comunes**). Cuando el vehículo llega a la planta se dirige al puesto con menos vehículos esperando y se queda ahí hasta que le dan el comprobante de la verificación. Los puestos atienden a los vehículos que van a él de acuerdo al orden de llegada pero dando prioridad a los vehículos sanitarios; cuando terminan de atender a un vehículo le debe entregar un comprobante con los detalles de la verificación. **Nota:** maximizar la concurrencia.

Resolver con **PASAJE DE MENSAJES ASINCRÓNICOS (PMA)** el siguiente problema. Se debe simular la atención en un banco con **3 cajas** para atender a **N clientes** que pueden ser **especiales** (son las embarazadas y los ancianos) o **regulares**. Cuando el cliente llega al banco se dirige a la caja con menos personas esperando y se queda ahí hasta que lo terminan de atender y le dan el comprobante de pago. Las cajas atienden a las personas que van a ella de acuerdo al orden de llegada pero dando prioridad a los clientes especiales; cuando terminan de atender a un cliente le debe entregar un comprobante de pago. **Nota:** maximizar la concurrencia.

Estos de arriba son lo mismo

PMS

Resolver con **PMS (Pasaje de Mensajes SINCRÓNICOS)** el siguiente problema. En una carrera hay ***C corredores*** y 3 ***Coordinadores***. Al llegar los corredores deben dirigirse a los coordinadores para que cualquiera de ellos le dé el número de “chaleco” con el que van a correr y luego se va. Los coordinadores atienden a los corredores de acuerdo al orden de llegada (cuando un coordinador está libre atiende al primer corredor que está esperando). ***Nota:*** maximizar la concurrencia.

```
Process Corredor[id: 0..C-1] {  
  int num;  
  
  AdminOrden ! quieroNumero(id);  
  Coordinador[*] ? tomaNumero(num);  
};
```

```
Process Coordinador[id: 0..2] {  
  int idC, num;  
  
  while (true) {  
    AdminOrden ! siguiente(id);  
    AdminOrden ? tomaCorredor (idC);  
    num = generarNumero();  
    Corredor[idC] ! tomaNumero(num);  
  };  
};
```

```
Process AdminOrden{  
  queue cola;  
  int idC, idE;  
  
  do Corredor[*] ? quieroNumero(idC) → push (cola, idC);  
    □ (not empty(cola)); Coordinador[*] ? siguiente(idE) →  
      pop(cola, idC);  
      Coordinador[idE] ! tomaCorredor(idC);  
  od  
};
```

Resolver con **PMS (Pasaje de Mensajes SINCRÓNICOS)** el siguiente problema. Simular la atención de una estación de servicio con un único surtidor que tiene ***un empleado*** que atiende a los ***N clientes*** de acuerdo al orden de llegada. Cada cliente espera hasta que el empleado lo atienda y le indica qué y cuánto cargar; espera hasta que termina de cargarle combustible y se retira. **Nota:** cada cliente carga combustible sólo una vez; todos los procesos deben terminar.

```
Process Cliente[id: 0..N-1] {
  AdminOrden ! solicitarPaso(id);
  Empleado ? pasar();
  Empleado ! cargar (tipo de combustible, cantidad);
  Empleado ? salir();
};
```

```
Process Empleado {
  int idC, cant;
  boolean terminar = false;
  text tipo;

  while (not terminar) {
    AdminOrden ! siguiente();
    AdminOrden ? tomaCliente (idC, terminar);
    Cliente[idC] ! pasar ();
    Cliente[idC] ? cargar (tipo, cant);
    // CargarCombustible(tipo, cant);
    Cliente[idC] ! salir ();
  };
};
```

```
Process AdminOrden{
  queue cola;
  int idC, idE, cantEnviados =0;
  boolean fin = false;

  while (not fin) {
    if Cliente[*] ? solicitarPaso(idC) → push (cola, idC);
    □ (not empty(cola)); Empleado ? siguiente() →
      cantEnviados++;
      if (cantEnviados == N) fin = true;
      pop(cola, idC);
      Empleado ! tomaCliente(idC, fin);
    fi
  };
};
```

Resolver con **PMS (Pasaje de Mensajes SINCRÓNICOS)** el siguiente problema. En una exposición aeronáutica hay un simulador de vuelo (que debe ser usado con exclusión mutua) y **un empleado** encargado de administrar el uso del mismo. A su vez hay ***P personas*** que van a la exposición y solicitan usar el simulador, cada una de ellas espera a que el empleado lo deje acceder, lo usa por un rato y se retira para que el empleado deje pasar a otra persona. El empleado deja usar el simulador a las personas respetando el orden en que hicieron la solicitud. ***Nota:*** cada persona usa sólo una vez el simulador.

```
Process Persona[id: 0..P-1] {  
  Empleado ! solicitarPaso(id);  
  Empleado ? pasar();  
  UsaSimulador;  
  Empleado ! salir();  
};
```

```
Process Empleado{  
  queue cola;  
  int idP;  
  bool libre = true;  
  
  do Persona[*] ? solicitarPaso(idP) →  
    if (not libre) push (cola, idP)  
    else { libre = false;  
          Persona[idP] ! pasar();  
        }  
  □ Persona[*] ? salir() →  
    if (empty(cola)) libre = true  
    else { pop(cola, idP);  
          Persona[idP] ! pasar();  
        }  
  od  
};
```


Resolver el siguiente problema con **Pasaje de Mensajes Sincrónicos (PMS)**. En una excursión hay una tirolesa que debe ser usada por **20 turistas**. Para esto hay un **guía** y un **empleado**. El empleado espera a que todos los turistas hayan llegado para darles una charla explicando las medidas de seguridad. Cuando termina la charla los turistas piden usar la tirolesa y esperan a que el guía les vaya dando el permiso de tirarse. El guía deja usar la tirolesa a un cliente a la vez y de acuerdo al orden en que lo van solicitando. **Nota:** todos los procesos deben terminar; suponga que el empleado tienen una función *DarCharla()* que simula que el empleado está dando la charla, y los turistas tienen una función *UsarTirolesa()* que simula que está usando la tirolesa.

Process Turista[id: 0..19] {

```
Empleado ! Llegada();
Empleado ? FinCharla();
Guia ! SolicitarUso(id);
Guia ? Pasar();
UsarTirolesa();
Guia ! Liberar();
};
```

Process Empleado {

```
int i;

for (i=0; i< 20; i++) Turista[*] ? Llegada();
DarCharla();
for (i=0; i< 20; i++) Turista[i] ! FinCharla();
};
```

Process Guia {

```
queue cola;
int idT, i;
bool libre = true;

for (i=0; i<40; i++) {
    if Turista[*] ? SolicitarUso (idT) →
        if (not libre) push (cola, idT)
        else { libre = false;
              Turista[idT] ! Pasar ();
              };
    □ Turista[*] ? Liberar () →
        if (empty(cola)) libre = true
        else { pop (cola, idT)
              Turista[idT] ! Pasar ();
              }
    fi
};
```

Semaforos

Resolver con **SEMAFOROS** el siguiente problema. En un examen final hay P alumnos y 3 profesores. Cuando todos los alumnos y los profesores han llegado comienza el examen. Para esto uno de los profesores (el primero en llegar) le da el examen a cada alumno. Cada alumno resuelve su examen, lo entrega y espera a que alguno de los profesores lo corrija y le indique la nota. Los profesores corrigen los exámenes respetando el orden en que los alumnos van entregando. **Nota:** maximizar la concurrencia.

```
sem mutexLlegada = 1;
sem mutexCola = 1;
sem primerProfesor = 0;
Sem esperaExamen[P] = ([P] 0);
sem hayExamen = 0;
sem esperaNota[P] = ([P] 0);
int notas[P], cant = 0, primerP = -1;
text exámenes[P];
queue cola;
```

```
Process Alumno[id: 0..P-1] {
    text examen;

    P(mutexLlegada);
    cant++;
    if (cant = P+3) V(primerProfesor);
    V(mutexLlegada);
    P(esperaExamen[id]);
    examen = resolver_examen(exámenes[id]);
    P(mutexCola)
    push(cola, (id, examen));
    V(mutexCola);
    V(hayExamen);

    P(esperaNota[id]);
    // puede acceder a la nota en notas[id]
};
```

```
Process Profesor [id: 0..2]{
    int i, idA;
    text e;

    P(mutexLlegada);
    cant++;
    if (cant = P+3) V(primerProfesor);
    if (primerP = -1) primerP = id;
    V(mutexLlegada);
    if (primerP == id) {
        P(primerProfesor);
        for (i=0; i<P; i++) {
            exámenes[i] = darExamen();
            V(esperaExamen[i]);
        };
    };

    while (true) {
        P(hayExamen);
        P(mutexCola);
        pop (cola, (idA, e));
        V(mutexCola)
        notas[idA] = corregir(e);
        V(esperaNota[idA]);
    }
};
```

Resolver el siguiente problema con **SEMÁFOROS**. En una competencia culinaria se juntan **1 jurado** y **C concursantes**. Una vez que todos los concursantes llegaron, el jurado les asigna la receta que deben realizar. A continuación, los concursantes cocinan el plato pedido (a cada uno le lleva un tiempo variable) y lo exhiben ante el jurado en el orden en que van terminando. El jurado asigna un puntaje a cada concursante, el cual lo guarda para su CV.

```
sem llegada=1;
sem esperar_concursantes=0;
sem jurado = 0;
sem esperar_receta[C] = ([C] 0);
sem esperar_nota[C] = ([C] 0);
sem mutex = 1;
sem presentar_plato = 0;
int cant = 0;
queue q;
recetas recetas[C];
platos platos[C];
int notas[N];
```

```
Process Concursante [id=1..C] {
    int i;
    P(llegada);
    cant++;
    if (cant < C) {
        V(llegada);
        P(esperar_concursantes);
    } else {
        V(llegada);
        V(jurado);
        for (i=0; i<C-1; i++)
            V(esperar_concursantes);
    }
    P(esperar_receta[id]);
    platos[id] = cocinar(receta[id]);
    P(mutex);
    push(q, id);
    V(mutex);
    V(presentar_plato);
    P(esperar_nota[id]);
    guardar_noto_en_CV(notas[id]);
}
```

```
Process Jurado {
    int i, idC;
    P(jurado);
    for (i=0; i<C; i++) {
        recetas[i] = generarReceta();
        V(esperar_receta[i]);
    }
    for (i=0; i<C; i++) {
        P(presentar_plato);
        P(mutex);
        idC = pop(q);
        V(mutex);
        notas[idC] = calificar(platos[idC]);
        V(esperar_nota[idC]);
    }
}
```


Resolver con **SEMÁFOROS** el siguiente problema. Se debe simular el uso de UNA máquina expendedora de agua con capacidad para 20 botellas por parte de U usuarios. Además existe **un repositor** encargado de reponer las botellas de la máquina. Cuando un usuario llega a la máquina expendedora, espera su turno (respetando el orden de llegada), saca una botella y se retira. Si encuentra la máquina sin botellas, le avisa al *repositor* para que cargue nuevamente la máquina con 20 botellas; espera a que se haga la recarga; saca una botella y se retira. **Nota:** maximizar la concurrencia; mientras se reponen las botellas se debe permitir que otros usuarios se encolen.

```
sem mutex = 1;
sem espera[C] = ([C] 0);
sem reponer = 0;
sem finReponer = 0;

bool libre = true;
queue colaMaq;
int cantBotellas = 20;
```

```
Process Repositor{
    while (true) {
        P(reponer);
        //reponer la máquina
        cantBotellas = 20;
        V(finReponer);
    };
};
```

```
Process Usuario[id: 0..U-1] {
    int i, auxU;

    P(mutex);
    if (not libre) { push(colaMaq, id);
                    V(mutex);
                    P(espera[id]);
                }
    else { libre = false;
          V(mutex);
        };
    if (cantBotellas == 0) {
        V(reponer);
        P(finReponer);
    };
    //saca una botella
    cantBotella--;
    P(mutex);
    if (empty(colaMaq)) libre = true
    else { pop(colaMaq, auxU);
          V(espera[auxU]);
        };
    V(mutex);
};
```

Resolver con **Semáforos** el siguiente problema. En una clínica hay un médico que debe atender a 20 pacientes de acuerdo al turno de cada uno de ellos (*no puede atender al paciente con turno $i+1$ si aún no atendió al que tiene turno i*). Cada paciente ya conocen su turno al comenzar (*valor entero entre 0 y 19, o entre 1 y 20, como les resulte más cómodo*), al llegar espera hasta que el médico lo llame para ser atendido, se dirige al consultorio y luego espera hasta que el médico lo termine de atender para retirarse. **Nota:** los únicos procesos que se pueden usar son los que representen a los pacientes y al médico; se debe asegurar que nunca haya más de un paciente en el consultorio; no se puede usar el ID del proceso como turno.

```
sem espera[20] = ([20] 0);  
sem entrada = 0, listo = 0, salida = 0;
```

```
Process Médico {  
    int turno;  
  
    for (turno=0; turno<20; turno++) {  
        V(espera[turno]);  
        P(entrada);  
        Atender_Paciente ();  
        V(listo);  
        P(salida);  
    };  
};
```

```
Process Paciente[id: 0..19] {  
    int miTurno = .....;  
  
    P(espera[miTurno]);  
    // Entar al consultorio  
    V(entrada);  
    // Espera a que lo terminen de atender  
    P(listo);  
    // Sale del consultorio  
    V(salida);  
}
```

Resolver con **Semáforos** el siguiente problema. En una casa de ropa, hay una modista que debe atender a 15 clientes que acuden para tomarse las medidas para realizarse un traje. Los clientes son atendidos de acuerdo al turno que tiene asignado cada uno de ellos (*no se puede atender al cliente con turno $i+1$ si aún no atendió al que tiene turno i*). Cada cliente ya conocen su turno al comenzar (*valor entero entre 0 y 14, o entre 1 y 15, como les resulte más cómodo*), al llegar espera hasta que la modista lo llame para ser atendido, se dirige al vestidor y espera hasta que la modista termina de tomarle las medidas para luego retirarse. **Nota:** los únicos procesos que se pueden usar son los que representen a los clientes y la modista; se debe asegurar que nunca haya más de un cliente en el vestidor; no se puede usar el ID del proceso como turno.

```
sem espera[15] = ([15] 0);  
sem entrada = 0, listo = 0, salida = 0;
```

```
Process Modista {  
    int turno;  
  
    for (turno=0; turno<15; turno++) {  
        V(espera[turno]);  
        P(entrada);  
        Tomar_Medidas();  
        V(listo);  
        P(salida);  
    };  
};
```

```
Process Cliente[id: 0..14] {  
    int miTurno = .....;  
  
    P(espera[miTurno]);  
    // Entar al vestidor  
    V(entrada);  
    // Espera a que lo terminen de medir  
    P(listo);  
    // Sale del vestidor  
    V(salida);  
}
```


Resolver con **Semáforos** el siguiente problema. En una sucursal de un banco, hay un empleado que debe atender a 20 clientes que acuden para averiguar sobre los productos del banco. Los clientes son atendidos de acuerdo al turno que tiene asignado cada uno de ellos (*no se puede atender al cliente con turno $i+1$ si aún no atendió al que tiene turno i*). Cada cliente ya conocen su turno al comenzar (*valor entero entre 0 y 19, o entre 1 y 20, como les resulte más cómodo*), al llegar espera hasta que la empleado lo llame para ser atendido, se dirige al escritorio y espera hasta que empleado termina de atenderlo para luego retirarse. **Nota:** los únicos procesos que se pueden usar son los que representen a los clientes y al empleado; se debe asegurar que nunca haya más de un cliente en el escritorio; no se puede usar el ID del proceso como turno.

```
sem espera[20] = ([20] 0);  
sem llegada = 0, listo = 0, salida = 0;
```

```
Process Empleado {  
    int turno;  
  
    for (turno=0; turno<20; turno++) {  
        V(espera[turno]);  
        P(llegada);  
        Atiende_Cliente();  
        V(listo);  
        P(salida);  
    };  
};
```

```
Process Cliente[id: 0..19] {  
    int miTurno = .....;  
  
    P(espera[miTurno]);  
    // Va al escritorio  
    V(llegada);  
    // Espera a que lo terminen de atender  
    P(listo);  
    // Se va del escritorio  
    V(salida);  
}
```

Resolver el siguiente problema con **SEMÁFOROS**. Simular la atención de una Planta Verificadora de Vehículos, donde se revisan cuestiones de seguridad de vehículos de a uno por vez. Hay ***N vehículos*** que deben ser verificados, donde algunos son autos y otros ambulancias. Antes de ser verificados, los autos deben hacer el pago correspondiente en la Caja de la Planta, donde le entregarán un recibo de pago. Las ambulancias no pagan. Los vehículos son atendidos de acuerdo al orden de llegada pero siempre dando prioridad a las ambulancias.

```
sem mutexCaja=1, mutexPlanta=1;
sem esperar_vehi=0;
sem esperar_verif[N] = ([N] 0);
sem caja = 0;
sem esperar_comprobante[N] = ([N] 0);
queue cola_ambus, cola_autos, cola_pagos;
comprobantes comps[N];
```

```
Process Vehiculo [id=1..N] {
    bool soy_auto = soy_Auto();

    if (soy_auto) {
        P(mutexCaja)
        push(cola_pagos, id);
        V(mutexCaja);
        V(caja);
        P(esperar_comprobante[id]);
    }
    P(mutexPlanta)
    if (soy_auto)
        push(cola_autos, id);
    else
        push(cola_ambus, id);
    V(mutexPlanta);
    V(esperar_vehi);
    P(esperar_verif[id]);
}
```

```
Proces Caja {
    int idV;
    while (true) {
        P(caja);
        P(mutexCaja);
        idV = pop(cola_pagos);
        V(mutexCaja);
        comps[idV] = generarComprobante();
        V(esperar_comprobante[idV]);
    }
}
```

```
Process Planta {
    int idV;

    while (true) {
        P(esperar_vehi);
        P(mutexPlanta);
        if (not (empty(cola_ambus)))
            idV = pop(cola_ambus);
        else
            idV = pop(cola_autos);
        V(mutexPlanta);
        // Verificar
        V(esperar_verif[idV]);
    }
}
```

Resolver con **SEMAFOROS** el siguiente problema. Una empresa de turismo posee UN micro con capacidad para 50 personas. Hay un único *vendedor* que atiende a los C *clientes* ($C > 50$) que intentan comprar un pasaje de acuerdo al orden de llegada (suponga que la atención de un cliente tarda un par de minutos), si aún hay lugares disponibles le indica el número de asiento que le tocó. Cada cliente, luego de ser atendidos por el vendedor, se dirige al micro para subir en caso de que le hayan dado un asiento, y en caso contrario se retira sin viajar. El micro espera a que los 50 pasajeros hayan subido para realizar el viaje. **Nota:** maximizar la concurrencia.

```
sem mutexCola = 1;
sem hayCliente = 0;
sem subioCliente = 0;
sem atendido[C] = ([C] 0);

int pasaje[C] = ([C] 0);
queue cola;
```

```
Process Cliente [id: 0..C-1] {
    int asiento;

    P(mutexCola);
    push(cola, id);
    V(mutexCola);
    V(hayCliente);

    P(atendido[id]);
    if (pasaje[id] > 0) V(subioCliente);
};
```

```
Process Micro {
    int i;

    for (i = 0; i < 50; i++) P(subioCliente);
    // Comienza el viaje
};
```

```
Process Vendedor {
    int cant = 0;
    int i, idC;

    for (i=0; i<C; i++) {
        P(hayCliente);
        P(mutexCola);
        pop(cola, idC);
        V(mutexCola);
        if (cant < 50) { cant ++;
                        pasaje[idC] = darAsiento();
                        };
        V(atendido[idC]);
    };
};
```


Resolver el siguiente problema con **SEMÁFOROS**. En un vacunatorio hay un empleado de salud para vacunar a 50 personas. El empleado de salud atiende a las personas de acuerdo al orden de llegada y de a 5 personas a la vez, es decir que cuando está libre debe esperar a que haya al menos 5 personas esperando, luego vacuna a las 5 primeras personas, y al terminar las deja ir para esperar por otras 5. Cuando ha atendido a las 50 personas el empleado de salud se retira.

Nota: todos los procesos deben terminar su ejecución; asegurarse de no realizar *Busy Waiting*; suponga que el empleado tienen una función *VacunarPersona()* que simula que el empleado está vacunando a UNA persona.

```
sem mutex = 1, atender = 0, listo = 0;
sem espera[50] = ([50] 0), irse[50] = ([50] 0);
queue c;
int cant = 0;
```

Process Persona [id: 0..49] {

```
    P (mutex);
    push (c, id);
    cant++;
    if (cant == 5) {
        cant = 0;
        V(atender);
    };
    V (mutex);
    P (espera[id]);
    V (listo);
    P (espera[id]);
    P (irse[id]);
};
```

Process Empleado {

```
    int i, j;
    int actuales[5];

    for (i=0; i<10; i++) {
        P (atender);
        P (mutex);
        for (j=0; j<5; j++) actuales[j] = pop (c);
        V (mutex);
        for (j=0; j<5; j++) {
            V(espera[actuales[j]]);
            P (listo);
            VacunarPersona(actuales[j]);
            V(espera[actuales[j]]);
        };
        for (j=0; j<5; j++) V(irse[actuales[j]]);
    };
};
```

Resolver con **SEMÁFOROS** el siguiente problema. Simular un examen escrito que deben rendir **60 alumnos** repartidos en **3 aulas** (20 alumnos en cada aula) con un docente en cada una de ellas. Cada alumno ya tiene asignado el aula en la que debe rendir. El docente de cada aula espera hasta que sus 20 alumnos hayan llegado para darles el enunciado del examen (el mismo para todos los alumnos), y luego les corrige el examen de acuerdo al orden en que van entregando. Cada alumno cuando llega debe esperar a que su docente (el correspondiente a su aula) le dé el enunciado del examen, lo resuelve, lo entrega para que el mismo docente lo corrija y le deje la nota. Cuando el alumno ya tiene su nota se retira. **Nota:** maximizar la concurrencia; sólo usar los procesos que representes a los alumnos y a los docentes; todos los procesos deben terminar.

Resolver con **SEMÁFOROS** el siguiente problema. Simular un examen técnico para concursos Nodocentes en la Facultad, en el mismo participan **100 personas** distribuidas en **4 concursos** (25 personas en cada concurso) con un coordinador en cada una de ellos. Cada persona ya conoce en que concurso participa. El coordinador de cada concurso espera hasta que lleguen las 25 personas correspondientes al mismo, les entrega el examen a resolver (el mismo para todos los de ese concurso), y luego corrige los exámenes de esas 25 personas de acuerdo al orden en que van entregando. Cada persona al llegar debe esperar a que su coordinador (el que corresponde a su concurso) le dé el examen, lo resuelve, lo entrega para que su coordinador lo evalúe y espera hasta que le deje la nota para luego retirarse. **Nota:** maximizar la concurrencia; sólo usar los procesos que representes a las personas y a los coordinadores; todos los procesos deben terminar.

```
text enunciados[3]; sem esperaP[3]=([3] 0), esperaE[3]=([3] 0);
sem mutex[3]=([3] 1), hayE[3]=([3] 0), hayN[60]=([60] 0);
int notas[60];      queue examenes[3];
```

Process Alumno[id: 0..59]

```
{ int miAula = ...;
  text miEx;

  V(esperaP[miAula]);
  P(esperaE[miAula]);
  miEx = ResolverExamen(enunciados[miAula]);
  P(mutex[miAula]);
  push(examenes[miAula], id, miEx);
  V(mutex[miAula]);
  V(hayE[miAula]);
  P(hayN[id]);
  --Accede a su nota en notas[id];
}
```

Process Docente[id: 0..2]

```
{ int i, idA;
  text exAlu;

  for (i=0; i<20; i++) P(esperaP[id]);
  enunciados[id] = ....;
  for (i=0; i<20; i++) V(esperaE[id]);
  for (i=0; i<20; i++) {
    P(hayE[id]);
    P(mutex[id]);
    pop(examines[id], idA, exAlu);
    V(mutex[id]);
    notas[idA] = Corregir(exAlu);
    V(hayN[idA]);
  }
}
```

ADA

Resolver el siguiente problema con **ADA**. En un negocio hay *un empleado* para atender los pedidos de clientes de *N clientes*; algunos clientes son *ancianos* o *embarazadas*. El empleado atiende los pedidos de acuerdo a la siguiente prioridad: primero las embarazadas, luego los ancianos, y por último el resto. Cada cliente hace sólo un pedido de la siguiente manera: en el caso de las embarazadas, si no son atendidas inmediatamente se retiran; en el caso de los ancianos, esperan a los sumo 5 minutos a ser atendidos, y si no se retira; cualquier otro cliente espera si o si hasta ser atendido. El empleado atiende a los clientes de acuerdo al orden de llegada pero manteniendo las siguientes prioridades: primero las embarazadas, luego los ancianos y luego el resto. **Nota:** suponga que existe una función *AtenderPedido()* que simula que el empleado está atendiendo a un cliente.

Procedure *Negocio* is

task type *Cliente*;

task *Empleado* is

entry pedE (p: IN text);

entry pedA (p: IN text);

entry pedR (p: IN text);

end *Empleado*;

arrCliente: array (0..N-1) of *Cliente*;

task body *Cliente* is

pedido: text :=

begin

if (*esEmbarazada()*) then

select

Empleado.pedE (pedido);

else

null;

end select;

elsif (*esAnciano()*) then

select

Empleado.pedA (pedido);

or delay 300.0

null;

end select;

else

Empleado.pedR (pedido);

end if;

end *Cliente*;

task body *Empleado* is

begin

loop

select

accept pedE (p: IN text) do

AtenderPedido(p);

end pedE;

or

when (pedE'count = 0) =>

accept pedA (p: IN text) do

AtenderPedido(p);

end pedA;

or

when (pedE'count = 0 and pedA'count = 0) =>

accept pedR (p: IN text) do

AtenderPedido(p);

end pedR;

end select;

end loop;

end *Empleado* ;

Begin

null;

End *Negocio*;

Resolver con **ADA** el siguiente problema. En una empresa están buscando nuevos empleados, para esto la Directora de Recursos Humanos tomará un examen escrito a 20 personas. Cuando una persona llega le da sus datos personales a la Directora para que ésta lo registre en el sistema (de acuerdo al orden de llegada). Cuando han llegado (y se han registrado) las 20 personas, la directora les entrega a todos el examen que deben resolver. Cuando una persona termina de resolver su examen se lo entrega a la Directora y espera a que ésta le dé la calificación. **Nota:** suponga que existen las funciones *Resolver_Examen* (llamada por las personas para simular la resolución del examen), *Corregir_Examen* (llamada por la Directora para simular la corrección de un examen) y *Registrar_Persona* (llamada por la Directora para simular que registra los datos de una persona en el sistema).

Procedure Examen is

```

task type Persona is
  entry darExamen (e: IN text);
end Persona;
task Directora is
  entry llegar (d: IN text);
  entry entregar (e: IN text; c: OUT integer);
end Directora;
arrP: array (0..19) of Persona;

task body Persona is
  nota: integer;
  datos: text := .....;
  examen: text;
begin
  Directora.llegar (datos);
  Accept darExamen (e: IN text) do
    examen := e;
  end darExamen;
  examen := Resolver_Examen (examen);
  Directora.entregar (examen, nota);
end Persona;
```

task body Directora is

```

  enunciado: text := .....;
begin
  for i in 0..19 loop
    accept llegar (d: IN text) do
      Registrar_Persona (d);
    end llegar;
  end loop;
  for i in 0..19 loop
    arrP(i).darExamen (enunciado);
  end loop;
  for i in 0..19 loop
    accept entregar(e: IN text; nota: OUT integer) do
      nota := Corregir_Examen (e);
    end entregar;
  end loop;
end Directora;
```

Begin

```

  null;
End Examen;
```

Resolver el siguiente problema con **ADA**. Hay un sitio web para identificación genética que resuelve pedidos de ***N clientes***. Cada cliente trabaja continuamente de la siguiente manera: genera la secuencia de ADN, la envía al sitio web para evaluar y espera el resultado; después de esto puede comenzar a generar la siguiente secuencia de ADN. Para resolver estos pedidos el sitio web cuenta con ***5 servidores*** idénticos que atienden los pedidos de acuerdo al orden de llegada (cada pedido es atendido por un único servidor). ***Nota:*** maximizar la concurrencia. Suponga que los servidores tienen una función *ResolverAnálisis* que recibe la secuencia de ADN y devuelve un entero con el resultado.

Procedure SitioWeb is

```

task type Cliente is
  entry Identificación (id: IN integer);
  entry Resultado (cod: IN integer);
end Cliente;
ArrC: array (0..N-1) of Cliente;

task type Servidor;
ArrS: array (0..4) of Servidor;

task Sitio is
  entry Pedido (sec: IN text, id: IN integer);
  entry Sig (s: OUT text, idC: OUT integer);
end Sitio;

task body Cliente is
  miID, res: integer;
  sec: text;
begin
  accept identificación (id: IN integer) do
    miID := id;
  end identificación
  loop
    sec = GeneraSecuenciaADN();
    Sitio.Pedido (sec, miID);
    accept Resultado (cod: IN integer) do
      res := cod;
    end Resultado;
    --usa el resultado guardado en res
  end loop;
end Cliente;

```

```

task body Servidor is
  sec: text;
  idC, codigo: integer;
begin
  loop
    Sitio.Sig (sec, idC);
    codigo := ResolverAnálisis(sec);
    ArrC(idC).Resultado(codigo);
  end loop;
end Servidor;

task body Sitio is
begin
  loop
    accept Sig (s: OUT text, idC: OUT integer) do
      accept Pedido (sec: IN text, id: IN integer) do
        s := sec;
        idC := id;
      end Pedido;
    end Sig;
  end loop;
end Sitio;

Begin
  for i in 0..N-1 loop
    ArrC(i).identificación(i);
  end loop;
End SitioWeb;

```

Resolver el siguiente problema con **ADA**. Existen ***N personas*** que van a realizar un pago en la caja de un banco. Las personas son atendidas de acuerdo al orden de llegada, aunque aquellos que sean jubilados tienen prioridad sobre los que no lo sean. Todas las personas esperan a lo sumo 15 minutos para ser atendidas. Si pasado ese tiempo no fueron atendidas, le dejan una nota de reclamo al responsable de Informes y se retiran.

Program Banco is

Task EmpleadoBanco IS

```
    entry atencionJub (t: IN Tramite; r: OUT Resultado);
    entry atencionNoJub (t: IN Tramite; r: OUT Resultado);
end EmpleadoBanco;
```

Task ResponsableInformes IS

```
    entry realizarReclamo(nota: IN Reclamo);
end ResponsableInformes;
```

Task Type Cliente;

TASK Body EmpleadoBanco IS

```
Begin
    loop
        SELECT
            WHEN (atencionJub'count == 0) =>
                Accept atencionNoJub (t: IN Tramite; r: OUT Resultado) do
                    r := resolverTramite(t);
                end atencionNoJub;
            OR
                Accept atencionJub (t: IN Tramite; r: OUT Resultado) do
                    r := resolverTramite(t);
                end atencionJub;
        End SELECT;
    End loop;
End EmpleadoBanco;
```

TASK Body ResponsableInformes IS

```
    reclamos: Queue;
Begin
    loop
        Accept realizarReclamo (n: IN Reclamo) do
            reclamos.push(n);
        end realizarReclamo;
    end loop;
End ResponsableInformes;
```

Arr_clientes: array (1..N) of Cliente;

TASK Body Cliente IS

```
    soy_jubilado: boolean;
    nota: Reclamo;
    tram: Tramite;
    res: Resultado;
Begin
    soy_jubilado := SoyJubilado();
    tram := generarTramite();
    if (soy_jubilado) then
        SELECT
            empleadoBanco.atencionJub (tram,res);
        OR DELAY(15*60);
        nota := generarNota();
        responsableInformes.realizarReclamo(nota);
        END SELECT;
    else
        SELECT
            empleadoBanco.atencionNoJub (tram,res);
        OR DELAY(15*60);
        nota := generarNota();
        responsableInformes.realizarReclamo(nota);
        end SELECT;
    end if;
End Cliente;
```

BEGIN

Null;

END Banco;

Resolver el siguiente problema con **ADA**. Se quiere modelar un puente colgante de un solo sentido, el cual resiste hasta 100 personas. Suponga que hay una cantidad innumerable de **personas** que quieren cruzar y que sólo lo podrán hacer si no superan la cantidad máxima. Las personas cruzan el puente de acuerdo al orden de llegada, aunque los jubilados tienen prioridad sobre los no jubilados.

Program Puente is

Task Type Jubilado;
Task Type NoJubilado;

Task Admin IS
 entry entrada_jubi();
 entry salida();
 entry entrada_nojubi();
end Admin;

arr_jubilados: array (1..A) of Jubilado;
Arr_nojubilados: array (1..B) of NoJubilado;

TASK Body Jubilado IS
Begin
 admin.entrada_jubi ();
 -- Pasar
 admin.salida();
End jubilado;

TASK Body NoJubilado IS
Begin
 admin.entrada_nojubi ();
 -- Pasar
 admin.salida();
End NoJubilado;

TASK Body Admin IS

cant: int := 0;

Begin

loop

 SELECT

 Accept salida ();
 cant--;

OR

 WHEN (cant < 100) => Accept entrada_jubi ();
 cant++;

OR

 WHEN ((cant < 100) AND (entrada_jubi'count == 0)) =>
 Accept entrada_nojubi ();
 cant++;

 END SELECT;

End loop;

End Admin;

Begin

 Null;

End Puente;

Resolver el siguiente problema con **ADA**. En una empresa de organización de recitales cuentan con un sitio web para la venta de entradas. Existen ***N personas*** que quieran comprar una de las E entradas disponibles. Las personas se dividen en pacientes e impacientes. Cuando las personas pacientes intentan hacer la compra en el sitio web, esperan a lo sumo 5 minutos a que el servidor responda. Si pasado ese tiempo no fueran atendidas, lo vuelven a intentar. En el caso de las personas impacientes, si no son atendidas inmediatamente por el servidor, esperan 10 segundos y vuelvan a intentarlo. En todos los casos, el sitio web les dice si pudieron venderle la entrada o no.

Procedure Sitio_Web is

Task Type ClientePaciente;
Task Type ClienteImpaciente;

Task SW IS
 entry atencion(r: OUT bool);
end SW;

clientesPacientes: array (1..N1) of ClientePaciente;
clientesImpacientes: array (1..N2) of ClienteImpaciente;

TASK Body ClientePaciente IS
 res: bool;
 atendido: bool = false;
Begin
 while (NOT atendido) loop
 SELECT
 sw.atencion(res);
 atendido := true;
 OR DELAY 300.0;
 null;
 end SELECT;
 end loop;

End ClientePaciente;

TASK Body ClienteImpaciente IS
 res: bool;
 atendido: bool = false;
Begin
 while (NOT atendido) loop

SELECT
 sw.atencion(res);
 atendido := true;
ELSE
 DELAY(10);
end SELECT;

end loop;

End ClienteImpaciente;

TASK Body SW IS

 cant_vendidas: int = 0;

Begin

 loop

 Accept atencion (r: OUT bool) do
 if (cant_vendidas <= E) then
 cant_vendidas++;
 r := true;
 else
 r:= false;
 end if;
 end atencion;

 end loop;

End SW;

Begin

 Null;

End SitioWeb;

Resolver el siguiente problema con **ADA**. Simular el funcionamiento de un Entrenamiento de Básquet donde hay **20 jugadores** y **UN entrenador**. El entrenador debe distribuir a los jugadores en 2 canchas. Cuando un jugador llega el entrenador le indica la cancha a la cual debe ir para que se dirija a ella y espere hasta que lleguen los 10; en ese momento comienzan a jugar el partido que dura 40 minutos. Cuando ambos partidos han terminado, el entrenador les da a los 20 jugadores una charla de 10 minutos y luego todos se retiran. El entrenador asigna el número de cancha en forma cíclica 1, 2, 1, 2 y así sucesivamente.

PROCEDURE Entrenamiento is

Task type Jugador;
arrJugadores: array (1..20) of jugador;

Task type Cancha is
entry llegue;
entry iniciar;
entry terminar;
End Cancha;
arrCanchas: array (1..2) of cancha;

Task Entrenador is
entry Asignar (numC: out integer);
entry TerminoPartido;
entry Salir;
End Entrenador;

Task Body Entrenador is

Begin

for C in 1..20 loop
accept Asignar (numC: out integer) do
numC = (C MOD 2) + 1;
end Asignar;
end loop;
for i in 1..2 loop accept TerminoPartido; end loop;
delay (10 minutos);
for C in 1..20 loop accept Salir; end loop;

End Entrenador;

Task Body Jugador is

numC: integer;

Begin

Entrenador.Asignar(numC);
arrCanchas(numC).llegue;
arrCanchas(numC).iniciar;
arrCanchas(numC).terminar;
Entrenador.Salir;

End Jugador;

Task Body Cancha is

Begin

for i in 1..10 loop accept llegue; end loop;
for i in 1..10 loop accept iniciar; end loop;
delay (60 minutos);
for i in 1..10 loop accept terminar; end loop;
Entrenador.TerminoPartido;

End Cancha;

Begin

Null;

END Entrenamiento;

Resolver el siguiente problema con **ADA**. Simular el funcionamiento de un Complejo de Canchas de Paddle que posee 10 canchas y donde hay **UN encargado** que se encarga de distribuir a las personas en las canchas. Al complejo acuden **40 personas** a jugar. Cuando una persona llega el encargado le indique el número de cancha a la cual debe ir, y se dirige a ella; cuando han llegado los 4 jugadores a la cancha, comienzan a jugar el partido que dura 60 minutos; al terminar el partido las 4 personas se retiran. El encargado asigna el número de cancha según el orden de llegada (los 4 primeros a la cancha 1, los siguientes 4 a la 2 y así sucesivamente). **Nota:** cuando el encargado ya atendió a las 40 personas debe terminar su ejecución.

PROCEDURE ComplejoPaddle is

Task type Jugador;
arrJugadores: array (1..40) of jugador;

Task type Cancha is

entry llegue;
entry iniciar;
entry terminar;

End Cancha;

arrCanchas: array (1..10) of cancha;

Task Encargado is

entry Asignar (numC: out integer);

End Encargado;

Task Body Encargado is

Begin

for C in 1..10 loop
for P in 1..4 loop
accept Asignar (numC: out integer) do
numC = C;
end Asignar;
end loop;
end loop;

End Encargado;

Task Body Jugador is

numC: integer;

Begin

Encargado.Asignar(numC);
arrCanchas(numC).llegue;
arrCanchas(numC).iniciar;
arrCanchas(numC).terminar;

End Jugador;

Task Body Cancha is

Begin

for i in 1..4 loop
accept llegue;
end loop;
for i in 1..4 loop
accept iniciar;
end loop;
delay (60 minutos);
for i in 1..4 loop
accept terminar;
end loop;

End Cancha;

Begin

Null;

END ComplejoPaddle;

Resolver con **ADA** el siguiente problema. Simular la venta de entradas a un evento musical por medio de un **portal web**. Hay **N clientes** que intentan comprar una entrada para el evento; los clientes pueden ser **regulares** o **especiales** (clientes que están asociados al sponsor del evento). Cada cliente especial hace un pedido al portal y espera hasta ser atendido; cada cliente regular hace un pedido y si no es atendido antes de los 5 minutos, vuelve a hacer el pedido siguiendo el mismo patrón (espera a lo sumo 5 minutos y si no lo vuelve a intentar) hasta ser atendido. Después de ser atendido, si consiguió comprar la entrada, debe imprimir el comprobante de la compra.

El portal tiene **E entradas** para vender y atiende los pedidos de acuerdo al orden de llegada pero dando prioridad a los Clientes Especiales. Cuando atiende un pedido, si aún quedan entradas disponibles le vende una al cliente que hizo el pedido y le entrega el comprobante.

Nota: no debe modelarse la parte de la impresión del comprobante, sólo llamar a una función Imprimir (comprobante) en el cliente que simulará esa parte; la cantidad **E** de entradas es mucho menor que la cantidad de clientes ($E \ll C$); todas las tareas deben terminar.

Procedure SitioWeb is

task *Portal* is

entry comprarE (comp: OUT text);

entry comprarR (comp: OUT text);

end *Portal*;

task type *Cliente*;

arrC: array (1..N) of *Cliente*;

task body *Cliente* is

c: text;

atendido: boolean := false;

begin

if (*esEspecial*) then

Portal.comprarE (c);

else

while (not atendido) loop

select

Portal.comprarR (c);

atendido := false;

or delay 300.0

null;

end select;

end loop;

end if;

if (c <> "") then *imprimir*(c);

end *Cliente*;

task body *Portal* is

com: text; vendidas: Integer:=0;

begin

for i in 1..N loop

Select

accept comprarE (comp: OUT text) do

if (vendidas < E) then

vendidas++;

comp := *hacerComprobante*;

else comp := "";

end if;

end comprarE;

or

when (comprarE'count = 0) => accept comprarR (comp: OUT text) do

if (vendidas < E) then

vendidas++;

comp := *hacerComprobante*;

else comp := "";

end if;

end comprarR;

end select;

end loop;

end *Portal*;

Begin

null;

End *SitioWeb*;

Resolver con **ADA** el siguiente problema. Simular la atención en una oficina donde atiende **un empleado**. Hay **N personas** que deben hacer UN trámite cada uno. Cuando una persona llega le da los datos al empleado y espera a que este resuelva el trámite y le entregue un comprobante. Por otro lado está el **director** de la oficina, el cual necesita 5 informes del empleado; cada vez que necesita un informe se lo pide al empleado y si no lo atiende inmediatamente sigue trabajando durante 10 minutos y luego lo vuelve a intentar siguiendo el mismo patrón (si no lo atiende inmediatamente trabaja durante 10 minutos y luego lo vuelve a intentar) hasta que el empleado lo atienda y le dé el informe pedido. El empleado atiende los pedidos de acuerdo al orden de llegada pero siempre dando prioridad a los pedidos de las personas. **Nota:** todas las tareas deben terminar.

Procedure Oficina is

```

task Empleado is
  entry tramite (D: IN text; comp: OUT text);
  entry informe (info: OUT text);
end Empleado;
task Director;
task type Cliente;
arrC: array (1..N) of Cliente;

task body Cliente is
  C,D: text;
begin
  D:= .....;
  Empleado.Tramite (D, C);
end Cliente;

task body Director is
  I: text;  cant: integer :=0;
begin
  while (cant<5) loop
    select
      Empleado.Informe(I);
      cant++;
    else
      delay (600.0)
    end select;
  end loop;
end Director;
```

task body Empleado is

```

  com: text;  vendidas: Integer:=0;
begin
  for i in 1..N+5 loop
    select
      accept tramite (D: IN text; comp: OUT text) do
        comp := ResolverTramite (D);
      end tramite;
    or
      when (tramite'count = 0) => accept informe (info: OUT text) do
        info := HacerInforme();
      end informe;
    end select;
  end loop;
end Empleado;

Begin
  null;
End Oficina;
```


Resolver con **ADA** el siguiente problema. Un *programador* contrató a **5 estudiantes** para testear los sistemas desarrollados por él. Cada estudiante tiene que trabajar con un sistema diferente y debe encontrar 10 errores (suponga que seguro existen 10 errores en cada sistema) que pueden ser: urgentes, importantes, secundarios. Cada vez que encuentra un error se lo reporta al programador y espera hasta que lo resuelva para continuar. El programador atiende los reportes de acuerdo al orden de llegada pero teniendo en cuenta las siguientes prioridades: primero los urgentes, luego los importantes y por último los secundarios; si en un cierto momento no hay reportes para atender durante 5 minutos trabaja en un nuevo sistema que está desarrollando. Después de resolver los 50 reportes de error (10 por cada estudiante) el programador termina su ejecución. **Nota:** todas las tareas deben terminar.

Procedure EmpresaSoftware is

```
task Programador is
  entry ErrorU (E: IN text; R: OUT text);
  entry ErrorI (E: IN text; R: OUT text);
  entry ErrorS (E: IN text; R: OUT text);
end Programador;
```

```
task type Estudiante;
arrE: array (1..5) of estudiante;
```

```
task body Estudiante is
  Err, Res, tipo: text;
begin
  for i in 1..10 loop
    EncontrarError ( Err, tipo);
    if (tipo = "Urgente") then
      Programador.ErrorU (Err, Res);
    elsif (tipo = "Importante") then
      Programador.ErrorI (Err, Res);
    elsif (tipo = "Secundario") then
      Programador.ErrorS (Err, Res);
    end if;
  end loop;
end Estudiante;
```

task body Programador is

```
  TE, TR: text;
  cant: integer:=0;
begin
  while (cant < 50) loop
    select
      accept ErrorU (E: IN text; R: OUT text) do
        R := ResolverError (E);
      end ErrorU; cant++;
    or when (ErrorU'count = 0) =>
      accept ErrorI (E: IN text; R: OUT text) do
        R := ResolverError (E);
      end ErrorI; cant++;
    or when (ErrorU'count = 0) && (ErrorI'count = 0) =>
      accept ErrorS (E: IN text; R: OUT text) do
        R := ResolverError (E);
      end ErrorS; cant++;
    else
      --Trabaja en otro Sistema
      delay (300.0);
    end select;
  end loop;
end Programador;
```

Begin

null;

End EmpresaSoftware;